

Migration toolkit for applications (MTA)

& Red Hat Developer Lightspeed for MTA Overview

Presenter's Name
Title

Presenter's Name
Title

Why Enterprises Modernize Applications

AI, emerging security threats, and cloud-native technology maturing has further motivated modernization



Lack of business agility and speed

Older technology makes it difficult to quickly innovate to meet new business and competitive challenges or revenue opportunities.



Security vulnerabilities

Old frameworks and applications can be more vulnerable to threats like data breaches, posing a threat to the business.



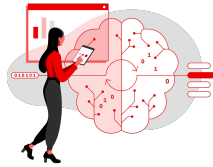
Increasing costs or cost-optimization

Older technology costs may increase, pushing enterprises to modernize. Or enterprises may be focused on lowering costs by improving application resource usage.



Drivers Further Motivating App Modernization

Rise of Generative AI



78%

.....

of Enterprises that are modernizing are either adding AI to legacy apps (42%) or are using it to modernize (53%)

Source: [Red Hat](#)

Software Supply Chain Security



742%

Average **annual increase in software supply chain attacks** over the past three years. **45%** of organizations will experience attacks. **Is a matter of when, not if.**

Source: [Sonatype](#)

Technical debt costs



~40%

.....

of IT balance sheets consists of technical debt, draining resources & slowing innovation.

Projects also cost 10-20% more due to tech debt.

Source: [Sonar](#)



Delivering Business Value Faster with Cloud-Native

Cloud-native technology led to cost savings, reduced development time, & reduced resource usage



"We've cut our application computing resource usage in three because the Knative mode of Red Hat Quarkus is so fast and so light."

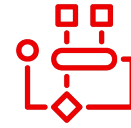
Steve Favez

Web Architect and Head of Internet Capability Center,
Etat du Valais



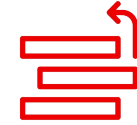
- ▶ Saved cost thanks to low memory and CPU consumption
- ▶ Reduced cluster usage by 40%, making room for new workloads
- ▶ Increased developer productivity threefold
- ▶ Hadeasy transition from Spring and J2EE for developers

Modernization Challenges



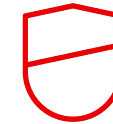
Legacy systems' complexity

Different migration variables pose risk of unintentionally altering or breaking application functionality.



Competing priorities

Need to balance meeting market challenges & opportunities with improving existing systems



Security concerns

May accidentally create new security vulnerabilities when modernizing an app



Key Elements to Accelerate App Modernization

People

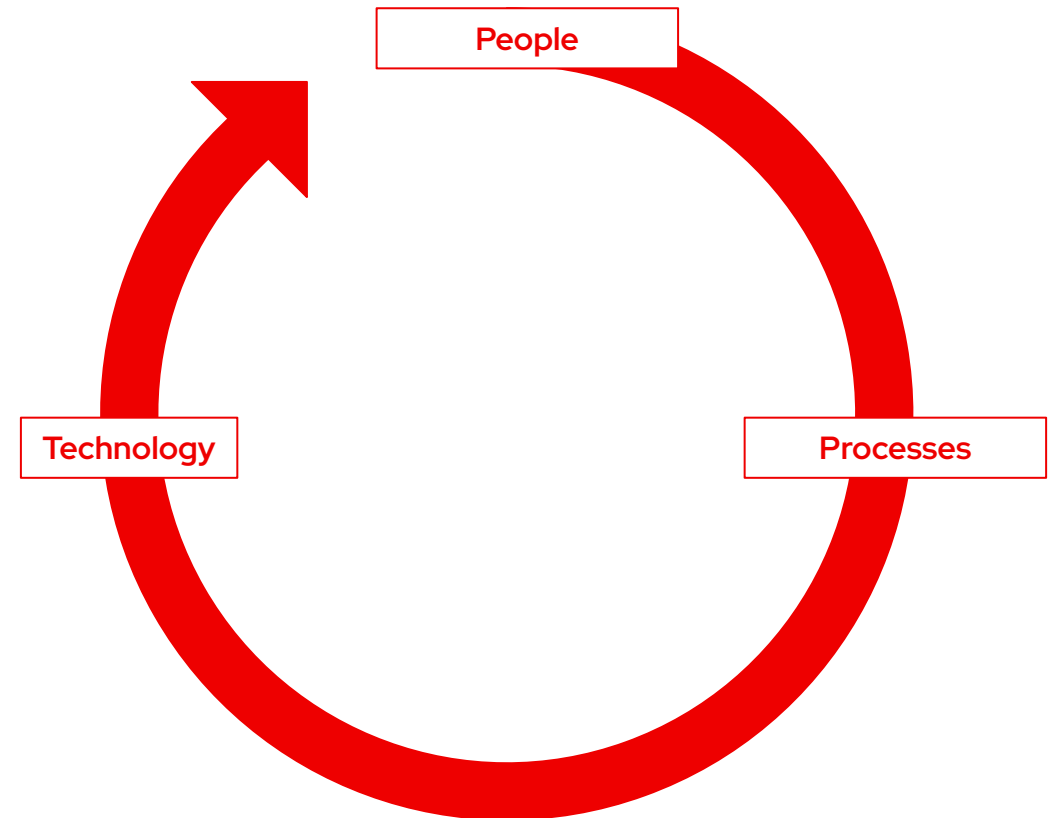
Project teams should collectively have expertise in the new technology, understand the enterprise's needs, and know how to drive application modernization initiatives.

Processes

Instead of attempting to modernize the whole portfolio at once, we recommend prioritizing a subset of existing applications, and upskilling staff on a team basis.

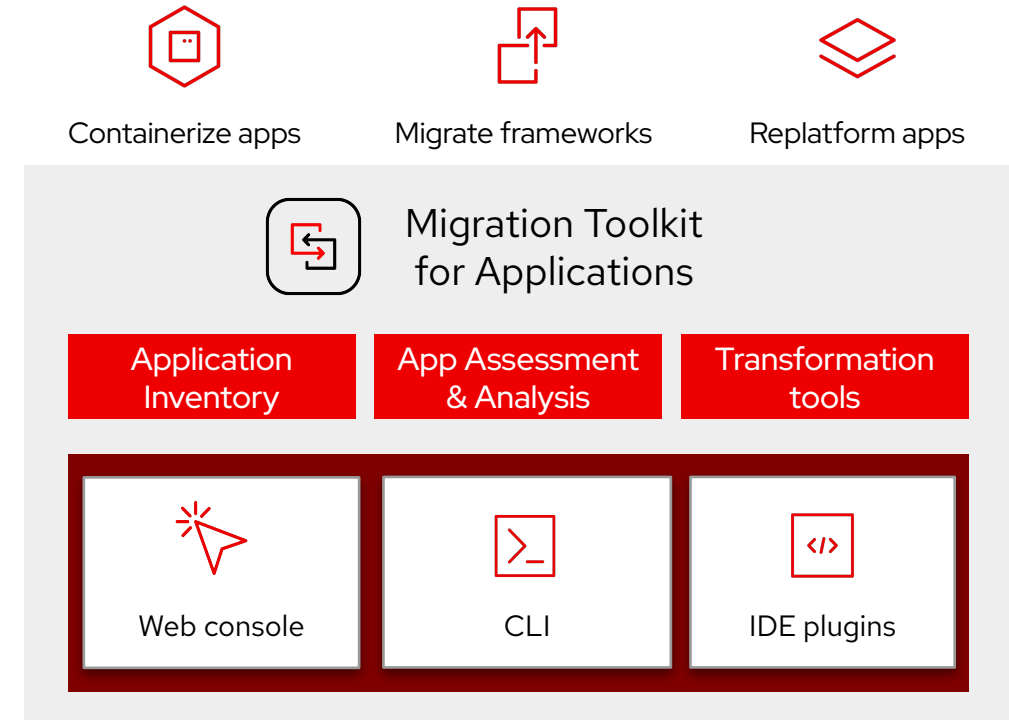
Technology

Organizations need an application platform that includes all the key capabilities required for both new app development and modernizing existing apps.



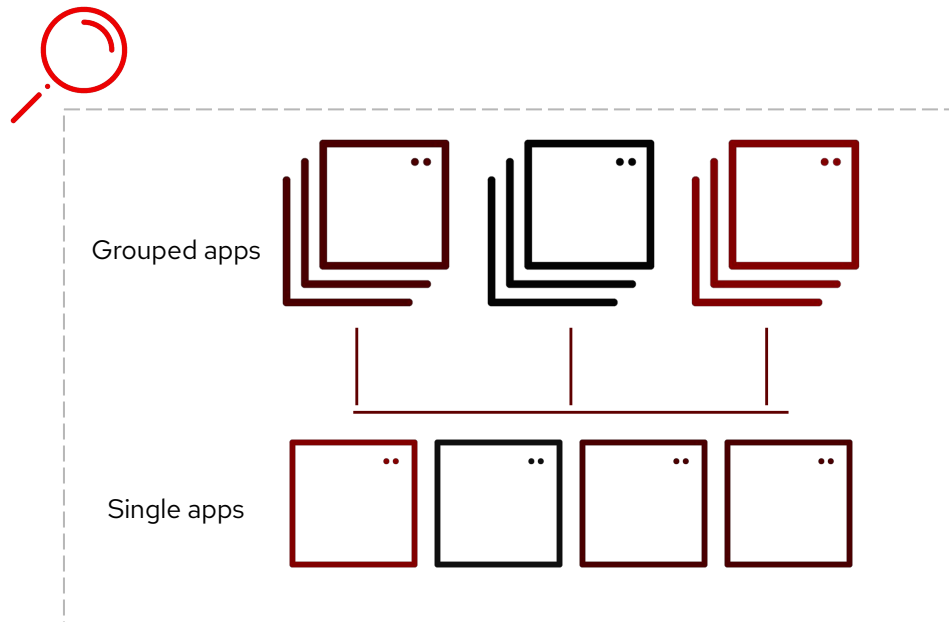
Modernize Existing Applications to Cloud-Native, Faster and Predictably

- ❑ **Decide which apps to modernize first & how to do it**
based on dependencies, potential issues, and estimated effort
- ❑ **Modernize faster** with guided or automated source code changes and automated app replatforming
- ❑ **Modernize safely** by equipping developers with pre-configured transformation tools to standardize migration execution



Decide Which Apps to Modernize First & How to Do It

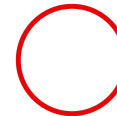
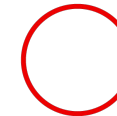
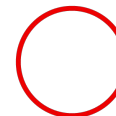
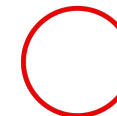
Get a comprehensive view of your portfolio, migration data, and groupings of similar apps



- Prioritize apps by seeing portfolio-level and single-application modernization details
- Determine effort and migration strategy for applications by identifying migration issues and dependencies with container suitability and source-code analysis
- Define project scope and efficiently migrate numerous apps by grouping similar applications together



What Are the Different Strategies for Modernization?

Strategy	Definition / Example	Time	Migration Cost	Operational Cost	Business Benefit
Retire	Sunsetting the application.				
Retain	Continue running the application as-is.				
Rehost	Migrating an application as-is to a new platform. <i>Use case: Migrating VMs to new platform, app server migration to cloud</i>				
Replatform	Making optimizations to the application that do not require re-architecture or significant code changes in order to achieve business or technical benefits. <i>Use case: Migrating an application into a container</i>				
Refactor	Changing how an application is developed and/or architected, typically to be more cloud-native. <i>Use case: Update the technology stack to cloud-enable an application</i>				
Repurchase	Moving to SaaS or Replacing portions of an application with as a service offerings. Example: Consuming Kafka as a Service within an existing application.				

MT
A

MT
A



Challenges of Replatforming Applications



Unsure which apps are on the older platform

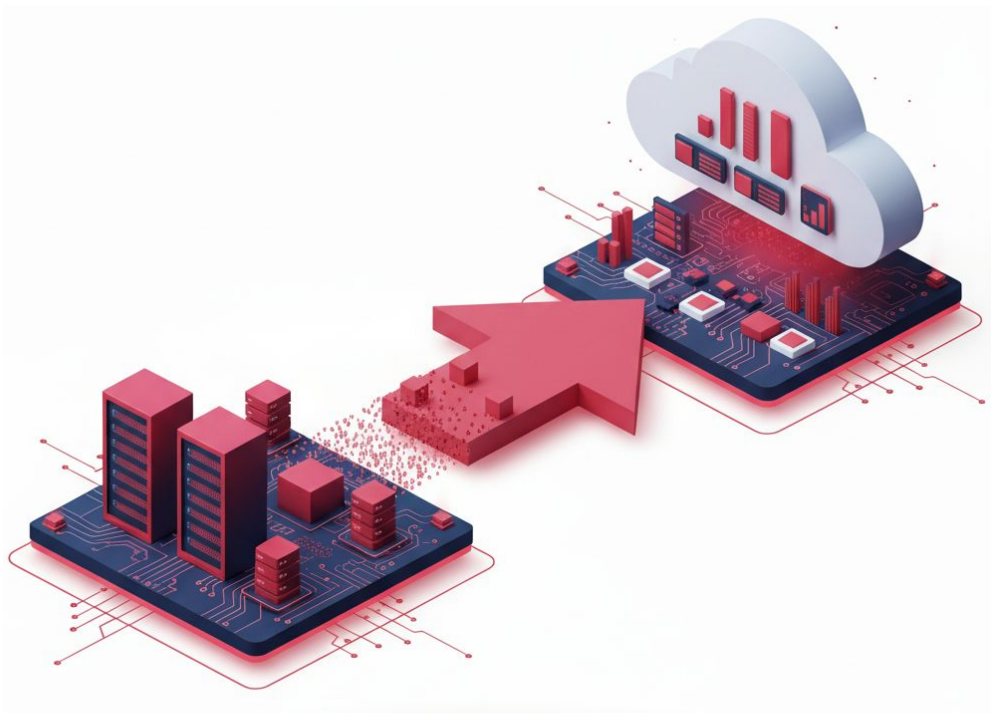


Creating deployment artifacts can be **filled** with manual, error-prone tasks



Hard to standardize deployment configuration across multiple developers





Automated Assets Generation

Replatform applications to OpenShift faster, lower cost

- ▶ **Automated application discovery and analysis:** Identify apps on source platform, extract deployment & runtime information
- ▶ **Generate deployment assets:** Use extracted information to auto-create deployment manifests for OpenShift
- ▶ **Streamline deployment:** Resulting artifacts are placed in target repository for automated deployment



Challenges of Refactoring Apps to Be Cloud-Enabled



Need **expertise** to know what code to change



Manually searching & fixing required code changes can be **time consuming**



Can be difficult to get automation tooling to migrate **custom frameworks and tech**



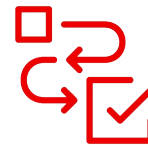
Speed up App Modernization with Guided Refactoring

Give developers guidance on migration issues and how to fix them



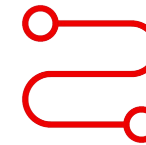
Automated code analysis

Analyze up to thousands of applications to find potential migration issues and issue locations within your code base



See detailed solutions

Get solutions to the found migration issues. You'll be able to see the suggested code and rationale behind it



Extendable migration paths

Use out-of-the-box migration paths or create your own migration path by specifying conditions and actions



Red Hat Developer Lightspeed for migration toolkit for applications



Accurate AI-generated code suggestions

Uses MTA's migration data to generate accurate code solutions that are tailored for your code



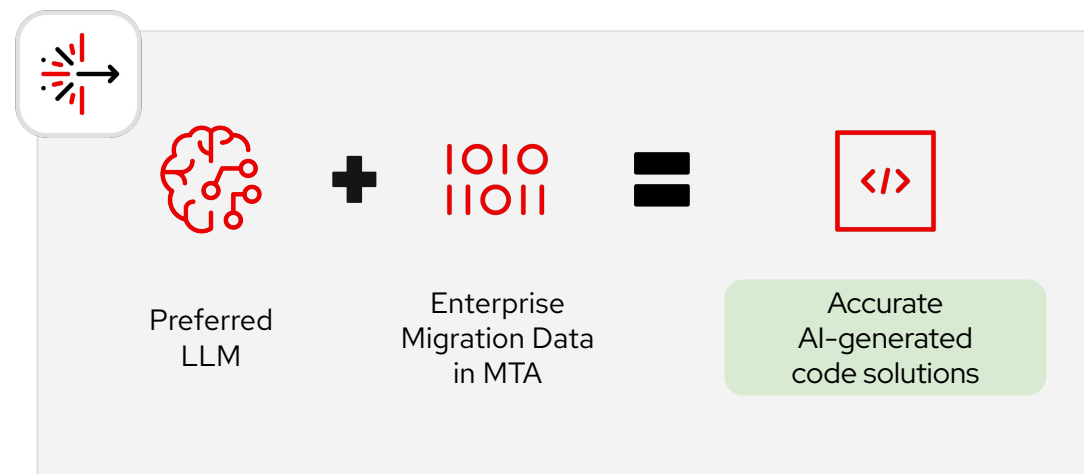
Apply code fixes with a click from IDE

Automatically apply code fixes to one or all occurrences of a migration issue within an app



Refactor similar applications efficiently

Once an application is migrated, code-change suggestions for similar apps become more accurate.



Modernize apps to cloud-native at scale with
intelligent app refactoring



Developer Lightspeed combines modernization approaches

Use code analysis to get AI models to provide accurate answers



Generative AI

Your preferred LLM generates and implements code fixes

+



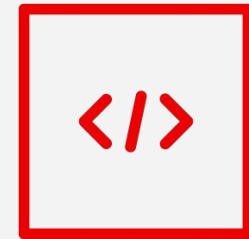
Enterprise Migration Data

stored in MTA

Rulesets-based code analysis identifies issues and solutions.

Remembers which code generations were successful.

=



Accurate AI-generated code

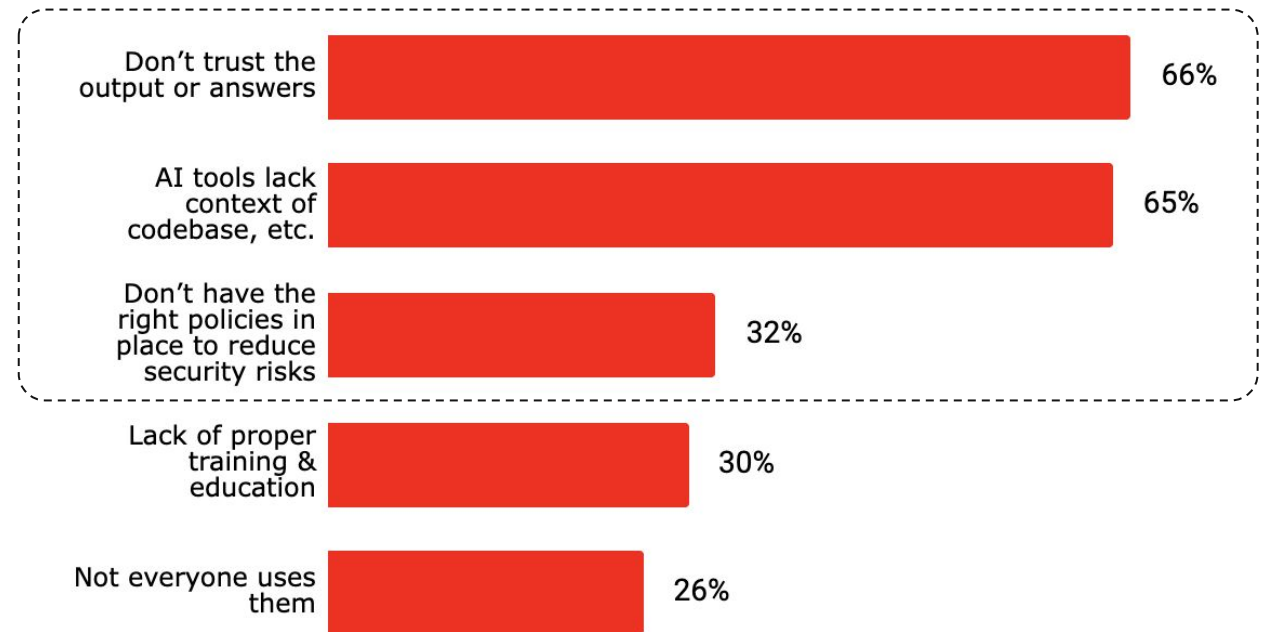
Migration data guides LLM's responses – leading to accurate code that improves over time.



Lightspeed for MTA Overcomes Common Challenges Developers Have with AI Tools By:

- Providing explanations for identified issues and suggested code solutions
- Using your codebase migration data that's stored in MTA to generate useful code solutions
- Allowing enterprises to bring their preferred public or self-hosted AI model

Challenges with AI at work¹



Refactoring with **AI Assistants**

Capabilities and constraints in an enterprise environment

Fantastic assistant for skilled developers

- **Easy to get started:** Uses natural-language prompts, making it accessible.
- **Comprehensive reasoning:** Applies logic derived from training data combined with an understanding of your code's intent.
- **Automates tedious work:** Efficiently handles syntax updates, dependency tracing, and code cleanup.

Difficult to refactor with it in an enterprise

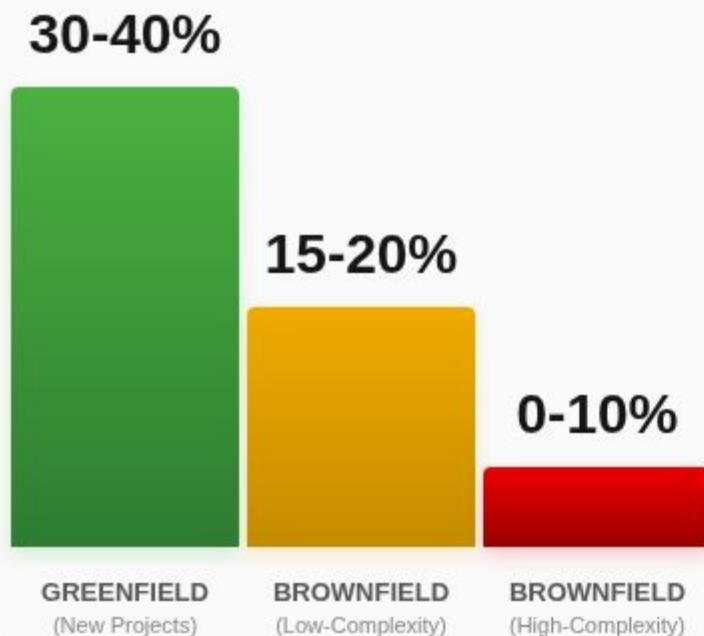
- **Still requires expertise:** Output varies widely based on user's prompting ability, requiring in-depth modernization knowledge to ensure quality.
- **Enterprise Knowledge Gaps:** Not trained on custom enterprise standards or able to scope the whole code base, leading to potential hallucinations, broken dependencies, or security vulnerabilities.
- **Rework and verification tax:** AI is ~50% less effective on brownfield apps. Developers spend more time fixing and auditing AI-generated code than if they would have manually written it.¹

1. Stanford Software Engineering Productivity Research Group: [Does AI Actually Boost Developer Productivity? \(100k Developers Study\)](#)

THE AI PRODUCTIVITY PARADOX

Stanford Software Engineering Productivity Study

PRODUCTIVITY GAINS BY ENVIRONMENT



THE KEY FINDING

While Generative AI delivers massive gains for new projects, its **effectiveness drops by ~50%** when navigating complex enterprise code



Context Fragmentation

AI can only view a tiny slice of the system at a time, it "hallucinates" or guesses about the parts it cannot see.



The Rework Loop

Developers spend more time auditing and fixing AI-generated "hallucinations" than they would have spent writing code manually.



Verification Tax

The mental effort required to ensure AI code meets strict corporate standards often exceeds the original time saved.

Supported Migration Paths

Replatforming

- ❑ Cloud Foundry to OpenShift
- ❑ Containerize existing apps (*start by using MTA's source-code change capabilities*)

Refactoring

- ❑ JBoss EAP 7 → 8
- ❑ Spring Framework 5 → Spring Framework 6
- ❑ Spring Boot 2 → Spring Boot 3
- ❑ Java 8 → 11, 17, 21
- ❑ Java EE/Jakarta EE → Quarkus
- ❑ Create-your-own to migrate custom tech



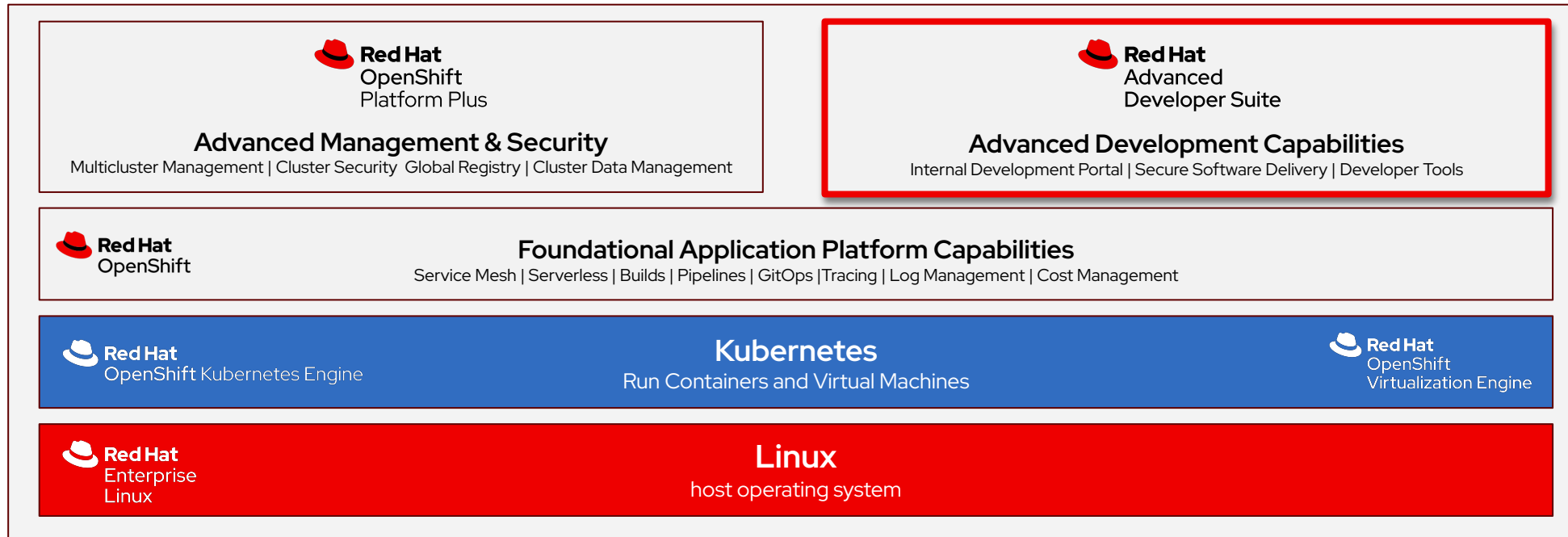


How do I get access to Red Hat Developer Lightspeed

- Advanced Developer Suite



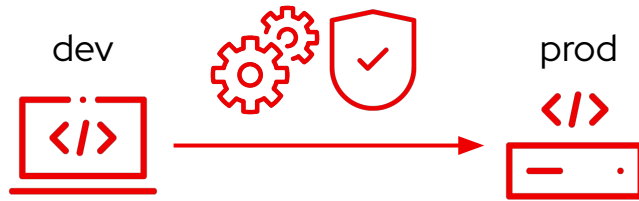
Developer Lightspeed is Part of Advanced Developer Suite



OpenShift Cloud Services

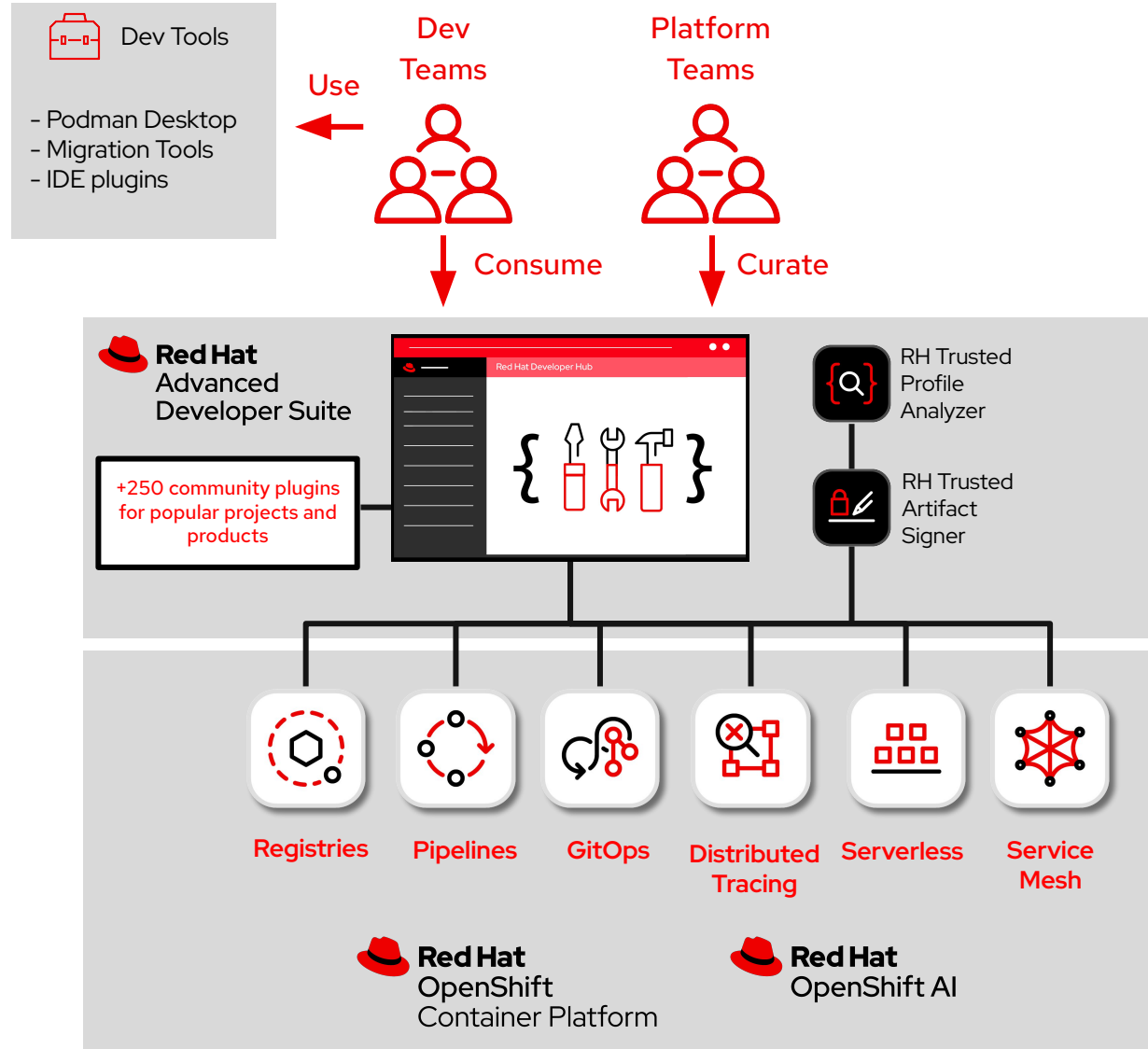


Red Hat Advanced Developer Suite



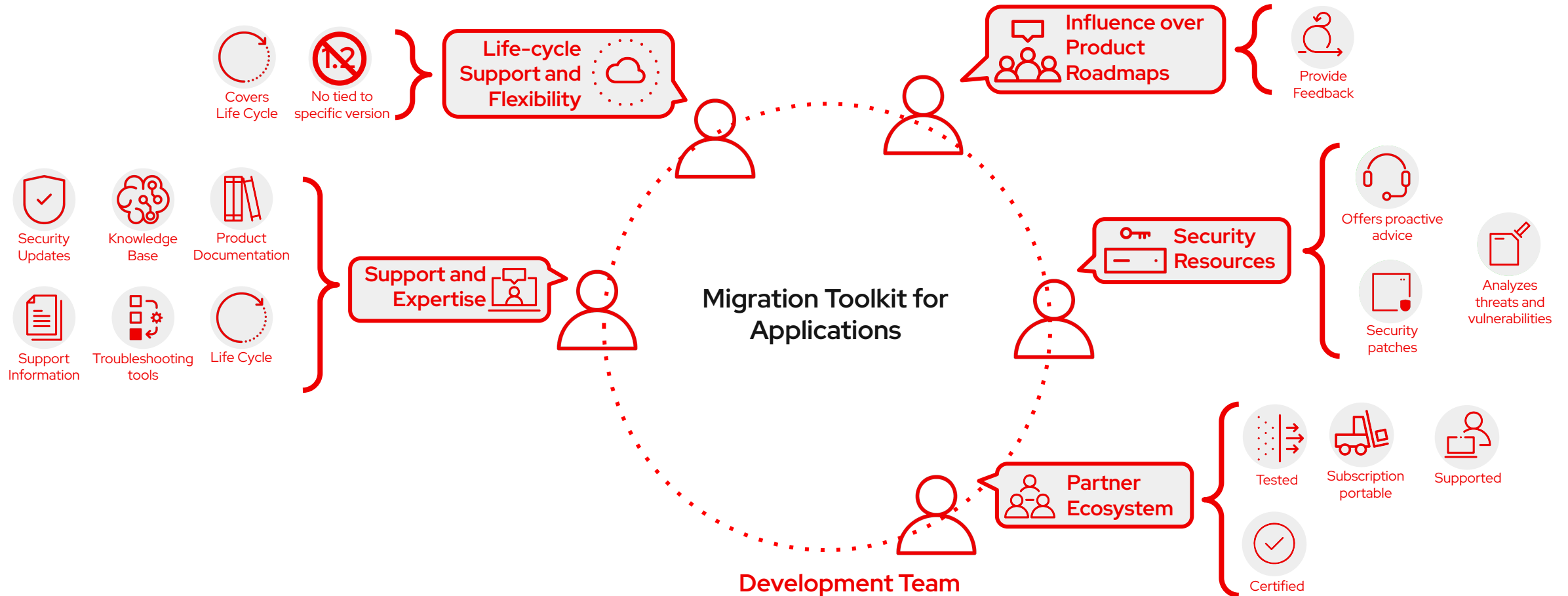
RHADS combines platform engineering tools with enhanced security capabilities to help accelerate and simplify application development with new enhancements to speed the adoption of AI

- Modular design allows it to be integrated with existing platforms and systems including Red Hat's platforms
- Integrates with existing security capabilities found in Red Hat Advanced Cluster Security
- Complements existing developer tools from Red Hat including OpenShift DevSpaces
- Built on leading open source projects like Backstage and Sigstore



Red Hat Subscription

Ensure your Development team is always delivering



Thank you

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.



linkedin.com/company/red-hat



youtube.com/user/RedHatVideos



facebook.com/redhatinc



x.com/RedHat